



21º CONGRESSO CIENTÍFICO

20º CONGRESSO DE INICIAÇÃO CIENTÍFICA PIBIC – CNPq | 4º WORKSHOP DE PROJETOS DE EXTENSÃO

FUNDAÇÃO HERMÍNIO OMETTO

"Desafios para Construção de um Mundo Sustentável"

08 E 09 | JUNHO

EVENTO
ONLINE
GRATUITO



**PARTICIPANTES
DE TODO O BRASIL**

REALIZAÇÃO:

Coordenadoria de
Comunidade e Extensão

FHO

FUNDAÇÃO
HERMÍNIO OMETTO



**CONSUMO DE API:
O QUE VOCÊ PRECISA SABER ANTES DE USAR**

ORLANDO SARAIVA JR

O que é API ?

API significa *Application Programming Interface* — mas a palavra mais importante aí é interface. Uma interface é um contrato entre dois lados: ela define o que pode ser pedido, como deve ser pedido e o que será devolvido, sem expor como funciona por dentro.

Pense numa tomada elétrica. Você não precisa saber como a usina gera energia, nem como os fios chegam até sua casa. Você só precisa saber que, se encaixar o plugue certo, recebe 127V. A tomada é a interface.

A API funciona exatamente assim: ela oculta a complexidade interna e expõe apenas o que o "cliente" precisa usar. Isso tem um nome em engenharia de software: encapsulamento. A API encapsula uma implementação e expõe uma superfície controlada de uso.

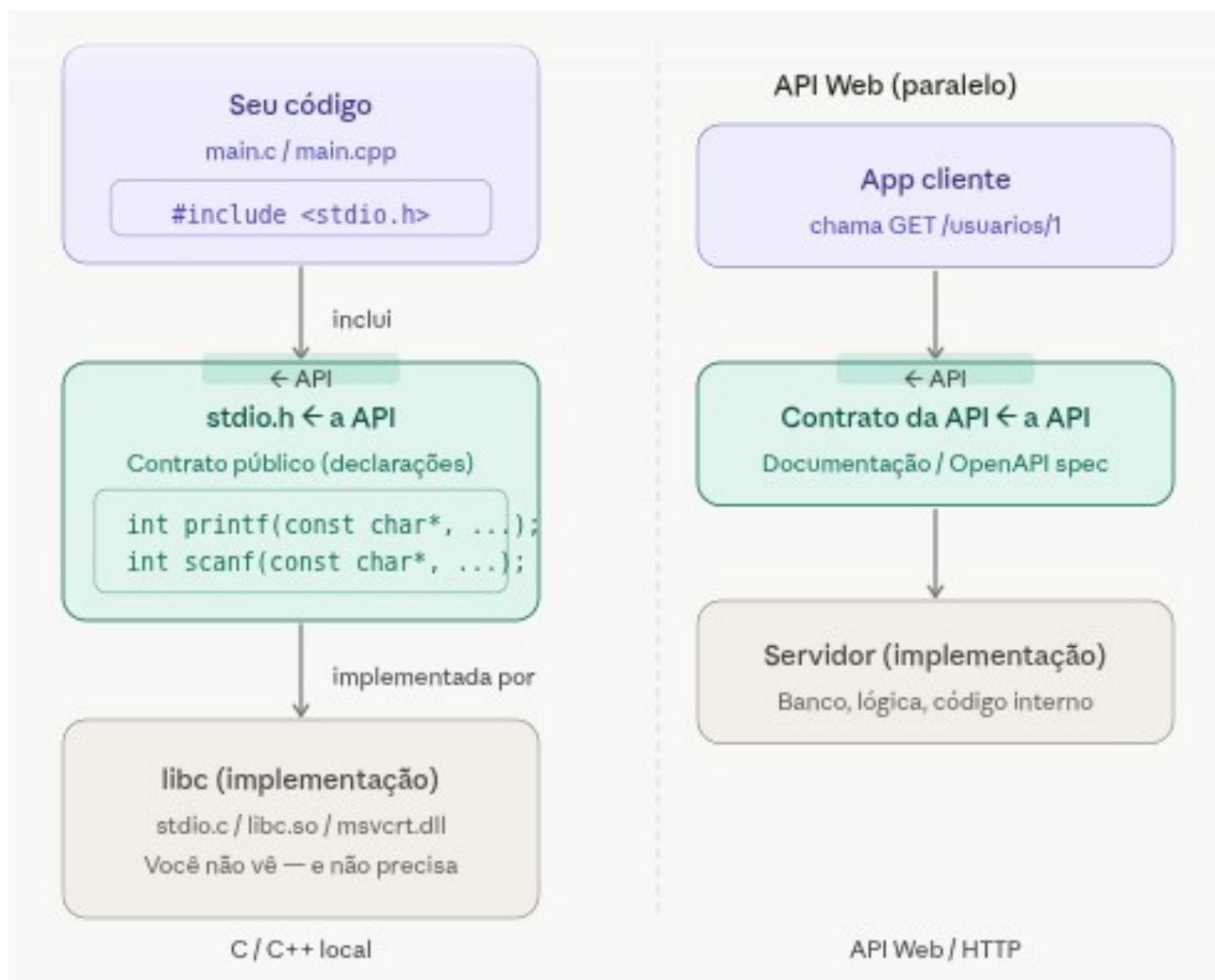
Os `#include` do C/C++ são APIs — e sempre foram

Quando você escreve `#include <stdio.h>`, está dizendo ao compilador: "quero ter acesso à interface da biblioteca de I/O padrão". Isso inclui `printf`, `scanf`, `fopen`, `fgets` e outras funções.

Você não tem o código-fonte de `printf` no seu projeto. Você não sabe (nem precisa saber) que internamente ela usa `write()` do kernel, formata buffers, lida com tipos variádicos via `va_list`.

Você só precisa saber a assinatura.

Application Programming Interface



API e Redes de Computadores

Para que essa comunicação funcione de forma confiável, organizada e independente de fabricante ou tecnologia, a indústria criou modelos de referência. O mais famoso deles é o Modelo OSI.

O Modelo OSI (Open Systems Interconnection) foi criado pela ISO no final dos anos 1970 e divide a comunicação em rede em sete camadas bem definidas. Cada camada tem uma responsabilidade clara e se comunica apenas com as camadas imediatamente acima e abaixo dela.

API e Redes de Computadores

As sete camadas do Modelo OSI:

Camada 7 — Aplicação: onde moram os protocolos que o usuário ou desenvolvedor enxerga diretamente (HTTP, FTP, SMTP, DNS).

Camada 6 — Apresentação: cuida da codificação, criptografia e compressão dos dados (TLS/SSL age aqui).

Camada 5 — Sessão: gerencia o estabelecimento, manutenção e encerramento de sessões de comunicação.

Camada 4 — Transporte: garante a entrega confiável dos dados ponta a ponta (TCP e UDP operam aqui).

Camada 3 — Rede: cuida do endereçamento lógico e roteamento dos pacotes (IP vive aqui).

Camada 2 — Enlace de Dados: trata da transmissão entre dispositivos na mesma rede local (Ethernet, Wi-Fi).

Camada 1 — Física: lida com bits brutos, cabos, sinais elétricos, ópticos ou de rádio.

Por que o Modelo OSI importa ?

Mesmo que você nunca configure um roteador ou analise pacotes de rede, entender o OSI ajuda a responder perguntas como:

- Por que minha requisição chegou ao servidor mas a resposta demorou? (pode ser latência de rede, camadas 1-3)
- Por que recebo um erro de SSL? (camada 6 — Apresentação / TLS)
- Por que o servidor recebeu dados corrompidos? (pode ser problema de transporte ou enlace)
- Por que minha API retorna 404? (camada 7 — lógica de aplicação)

Cada tipo de problema tem um "endereço" nas camadas OSI. Saber disso economiza horas de depuração.

Modelo TCP/IP

Enquanto o OSI é um modelo teórico de referência, o TCP/IP é o conjunto de protocolos que realmente faz a Internet funcionar. Na prática, ele é frequentemente descrito em quatro camadas:

Camada de Acesso à Rede (equivalente às camadas 1–2 do OSI): Ethernet, Wi-Fi, fibra óptica.

Camada de Internet (equivalente à camada 3 do OSI): protocolo IP, roteamento de pacotes.

Camada de Transporte (equivalente à camada 4 do OSI): TCP e UDP.

Camada de Aplicação (equivalente às camadas 5–7 do OSI): HTTP, DNS, SMTP, etc.

Protocolo TCP

O TCP (*Transmission Control Protocol*) opera na camada de transporte e é responsável por garantir que os dados cheguem corretamente, na ordem certa e sem perdas.

Para isso, ele usa um mecanismo chamado three-way handshake (aperto de mão de três vias):

SYN: o cliente envia um pacote de sincronização ao servidor, dizendo "quero me conectar".

SYN-ACK: o servidor responde confirmando a conexão.

ACK: o cliente confirma o recebimento, e a conexão está estabelecida.

TCP ou UDP ?

Nem toda comunicação precisa de confiabilidade garantida. O UDP (User Datagram Protocol) é mais rápido e leve, mas não garante entrega nem ordem dos pacotes. Ele é usado em aplicações onde velocidade é mais importante que perfeição, como streaming de vídeo ao vivo, jogos online e DNS.

Para APIs REST tradicionais — que é o foco desta palestra — o TCP é sempre a base, garantindo que sua resposta JSON chegue inteira e na ordem correta.

O protocolo HTTP

HTTP (Hypertext Transfer Protocol) é o protocolo da camada de aplicação responsável pela comunicação na Web. É através dele que navegadores, aplicativos e serviços trocam recursos: páginas HTML, imagens, JSON, XML, vídeos, e muito mais.

Quando você consome uma API REST, você está usando HTTP. Toda a lógica que o desenvolvedor escreve — as rotas, os parâmetros, os cabeçalhos de autenticação, o formato do corpo da requisição — existe na camada de aplicação, sobre o HTTP. As camadas abaixo (TCP, IP, Ethernet) são transparentes para você.

Anatomia de uma requisição HTTP

Uma requisição HTTP é composta por:

Método (verb): define a intenção da operação. Os mais comuns são GET (buscar), POST (criar), PUT/PATCH (atualizar) e DELETE (remover).

URL: o endereço do recurso.

Ex.: `https://api.exemplo.com/v1/usuarios/42`

Headers (cabeçalhos): metadados da requisição, como Content-Type, Authorization, Accept.

Body (corpo): dados enviados ao servidor, geralmente em formato JSON. Presente em POST, PUT e PATCH.

Anatomia de uma resposta HTTP

Uma resposta HTTP contém:

Status Code: um número de 3 dígitos indicando o resultado. Ex.: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.

Headers: metadados da resposta, como Content-Type, Cache-Control, X-Rate-Limit.

Body: os dados devolvidos pelo servidor, quase sempre em JSON nas APIs modernas.

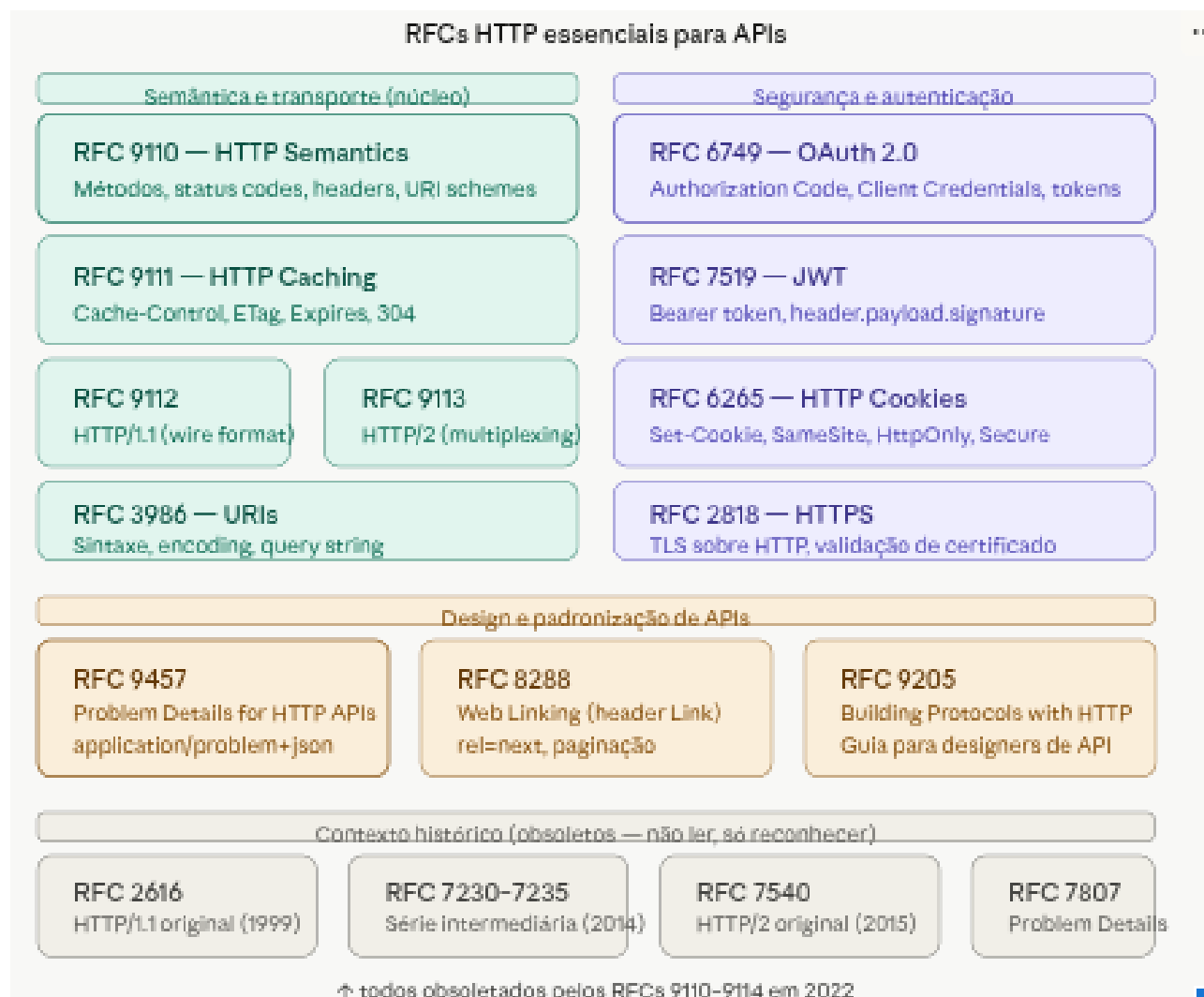
RFCs

RFC significa Request for Comments — são documentos técnicos publicados pelo IETF (Internet Engineering Task Force) que definem os padrões, protocolos e convenções que sustentam a Internet.

O nome é histórico e um pouco enganoso: apesar de "pedir comentários", uma RFC publicada é na prática uma especificação normativa e oficial — é o documento que diz exatamente como o HTTP, o TCP, o DNS, o e-mail e dezenas de outros protocolos devem funcionar.

Qualquer fabricante, sistema operacional ou linguagem de programação que queira ser interoperável precisa seguir o que está escrito ali.

RFCs importantes





HANDS ON

Boas Práticas ao consumir API

Segurança

- Nunca exponha chaves de API em repositórios públicos. Use `.gitignore` para excluir arquivos `.env`.
- Sempre use HTTPS. Jamais transmita tokens por HTTP puro.
- Implemente timeout nas suas requisições para evitar que sua aplicação trave aguardando uma resposta.
- Valide e sanitize os dados recebidos da API antes de usá-los.

Tratamento de erros

- Trate explicitamente os códigos de status HTTP — não assuma que toda resposta é 200.
- Implemente retry com backoff exponencial para erros temporários (429, 503).
- Diferencie erros de cliente (4xx) de erros de servidor (5xx) na sua lógica de tratamento.
- Loggue detalhes dos erros para facilitar o diagnóstico em produção.

Boas Práticas ao consumir API

Performance

- Reutilize conexões TCP usando Session (Python requests) ou conexões persistentes (Java HttpClient).
- Implemente cache para dados que não mudam frequentemente (ex.: lista de países, moedas).
- Respeite os headers de rate limit (X-Rate-Limit, Retry-After) para evitar bloqueios.
- Prefira chamadas assíncronas (async/await) quando precisar fazer múltiplas requisições simultâneas.

Documentação e Contratos

- Leia sempre a documentação oficial da API antes de integrá-la.
- Verifique a versão da API — URLs com /v1/, /v2/ indicam versões, e comportamentos podem mudar entre elas.
- Fique atento a datas de depreciação de endpoints e versões antigas.



FUNDAÇÃO
HERMÍNIO OMETTO

SÍNTESE

O que você precisa saber

1. Fundamentos de lógica e estrutura de dados Antes de qualquer coisa, você precisa se sentir confortável com variáveis, condicionais, laços e funções na linguagem que escolher. Mas o ponto específico para APIs é entender dicionários/mapas (o equivalente ao objeto JSON na sua linguagem), porque praticamente toda resposta de API moderna chega como JSON — uma estrutura de chave-valor aninhada. Saber navegar, iterar e transformar essas estruturas é o dia a dia do consumo de APIs.

2. HTTP na prática Você não precisa decorar RFCs, mas precisa entender conceitualmente os métodos (GET busca, POST cria, PUT/PATCH atualiza, DELETE remove), os status codes (o que diferencia um 200 de um 201, por que 401 é diferente de 403, o que fazer com um 429 ou 503) e a estrutura de uma requisição: URL, headers e body. Sem isso, você estará tentando resolver problemas às cegas.

O que você precisa saber

3. Serialização e desserialização de JSON Na prática: saber converter um objeto da sua linguagem para JSON (serializar) e JSON de volta para um objeto (desserializar/parsear). Em Python é `json.loads()` e `json.dumps()`, ou simplesmente `response.json()` com a biblioteca `requests`. Em Java é usar Jackson ou Gson. Em Ruby é `JSON.parse()`. Parece trivial, mas erros de tipo (número chegando como string, campo nulo não tratado, data em formato inesperado) são responsáveis por uma fatia enorme dos bugs em integrações.

4. Tratamento de erros Esse é o ponto que a maioria dos tutoriais ignora e onde os sistemas quebram em produção. Você precisa saber verificar o status code antes de processar a resposta, tratar exceções de rede (timeout, conexão recusada, DNS falhou), lidar com respostas inesperadas (a API retornou HTML em vez de JSON porque você bateu numa página de erro), e implementar retry com backoff exponencial para erros temporários. Uma integração sem tratamento de erros funciona em ambiente de desenvolvimento e quebra no primeiro imprevisto em produção.

O que você precisa saber

5. Autenticação A maioria das APIs reais exige autenticação. Você precisa entender pelo menos três mecanismos: API Key (um token fixo enviado no header ou query string), Bearer Token (um token dinâmico, geralmente JWT, enviado no header Authorization: Bearer <token>) e o fluxo básico do OAuth 2.0 (quando você precisa agir em nome de um usuário). Além disso, precisa saber guardar essas credenciais com segurança — variáveis de ambiente, nunca no código-fonte.

6. Leitura de documentação Essa é uma habilidade subestimada e completamente separada de saber programar. Você precisa saber ler uma referência de API e extrair: qual é o endpoint, quais parâmetros são obrigatórios vs opcionais, qual é o formato esperado do body, o que cada campo da resposta significa e quais são os possíveis erros. Documentações como Swagger/OpenAPI, Postman Collections ou páginas de referência da Stripe, GitHub ou Twilio são formatos que você vai encontrar repetidamente.

Orlando Saraiva Jr

<https://github.com/orlandosaraivajr>



FUNDAÇÃO
HERMÍNIO OMETTO



FUNDAÇÃO
HERMÍNIO OMETTO

Cursos **Livres**

Cursos de Curta e Média Duração

*Desenvolver habilidades
é investir no futuro*

➤ inscrições abertas

